

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
!nvidia-smi
```

Wed Sep 23 17:18:21 2026

```
+-----+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----+-----+-----+-----+-----+
| GPU  Name           Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.         |
+-----+-----+-----+-----+-----+
|    0   Tesla T4              Off      | 00000000:00:04.0 Off |                    0 |
| N/A   44C    P8              10W / 70W |  0MiB / 15360MiB |      0%      Default |
+-----+-----+-----+-----+-----+
| Processes: |
| GPU   GI   CI          PID    Type   Process name                  GPU Memory |
|   ID   ID                 |                   |            Usage         |
+-----+-----+-----+-----+-----+
| No running processes found               |
+-----+-----+-----+-----+-----+
```

```
!pip install -U transformers datasets peft trl bitsandbytes accelerate
```

[Show hidden output](#)

```
import torch

from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    BitsAndBytesConfig
)

from peft import LoraConfig
```

```
print(torch.cuda.is_available())
```

True

```
!pip install -U datasets
```

[Show hidden output](#)

```
from datasets import load_dataset

dataset = load_dataset("stanfordnlp/imdb")
```

[Show hidden output](#)

```
print(dataset)
```

```
DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  unsupervised: Dataset({
    features: ['text', 'label'],
    num_rows: 50000
  })
})
```

```
print(dataset["train"][0])
```

```
{'text': 'I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that surrounded it when it was fir
```

```
print(dataset["train"].features)
```

```
{'text': Value('string'), 'label': ClassLabel(names=['neg', 'pos'])}
```

```
train_dataset = dataset["train"].shuffle(seed=42).select(range(2000))
test_dataset = dataset["test"].shuffle(seed=42).select(range(500))
```

```
print("Training:", len(train_dataset))
print("Testing:", len(test_dataset))
```

```
Training: 2000
Testing: 500
```

```
def format_example(example):
    sentiment = "Positive" if example["label"] == 1 else "Negative"

    return {
        "text": f"""Review:
{example["text"]}

Sentiment:
{sentiment}"""
    }
```

```
train_dataset = train_dataset.map(format_example)
test_dataset = test_dataset.map(format_example)
```

```
Map: 100%                               2000/2000 [00:00<00:00, 5528.51 examples/s]
Map: 100%                               500/500 [00:00<00:00, 6512.57 examples/s]
```

```
print(train_dataset[0]["text"])
```

```
Review:
There is no relation at all between Fortier and Profiler but the fact that both are police series about violent crimes. Prof

Sentiment:
Positive
```

```
!pip install -U transformers peft trl bitsandbytes accelerate
```

[Show hidden output](#)

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
```

```
print(torch.cuda.is_available())
```

```
True
```

```
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True
)
```

```
model_name = "Qwen/Qwen2.5-1.5B-Instruct"

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto"
)
```

[Show hidden output](#)

```
tokenizer = AutoTokenizer.from_pretrained(model_name)

tokenizer.pad_token = tokenizer.eos_token
```

[Show hidden output](#)

```
print("Tokenizer loaded successfully!")
print("Pad token:", tokenizer.pad_token)
```

```
Tokenizer loaded successfully!
Pad token: <|im_end|>
```

```
from peft import prepare_model_for_kbit_training

model = prepare_model_for_kbit_training(model)

print("Model prepared for QLoRA training!")
```

Model prepared for QLoRA training!

```
from peft import LoraConfig

lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=[
        "q_proj",
        "k_proj",
        "v_proj",
        "o_proj"
    ]
)

print("LoRA configuration created!")
```

LoRA configuration created!

```
import trl
print("TRL version:", trl.__version__)
```

TRL version: 1.13.0

```
from trl import SFTConfig

training_args = SFTConfig(
    output_dir="./qwen_imdb_qlora",
    num_train_epochs=1,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    logging_steps=10,
    save_steps=100,
    fp16=True,
    report_to="none",
    max_length=512
)

print("Training configuration created!")
```

Training configuration created!

```
from trl import SFTTrainer

trainer = SFTTrainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    processing_class=tokenizer,
    peft_config=lora_config,
)

print("SFT Trainer created successfully!")
```

```
Adding EOS to train dataset: 100%                2000/2000 [00:00<00:00, 10943.10 examples/s]
Tokenizing train dataset: 100%                   2000/2000 [00:02<00:00, 886.22 examples/s]
Building labels for train dataset: 100%           2000/2000 [00:00<00:00, 3044.75 examples/s]
Truncating train dataset: 100%                   2000/2000 [00:00<00:00, 2510.59 examples/s]
Dropping fully masked examples from train dataset: 100% 2000/2000 [00:00<00:00, 2417.17 examples/s]
SFT Trainer created successfully!
```

```
import torch
```

```
print("BF16 supported:", torch.cuda.is_bf16_supported())
print("FP16 available:", torch.cuda.is_available())
```

```
BF16 supported: True
FP16 available: True
```

```
from trl import SFTConfig

training_args = SFTConfig(
    output_dir="./qwen_imdb_qlora",
    num_train_epochs=1,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    logging_steps=10,
    save_steps=100,
    bf16=True,
    fp16=False,
    report_to="none",
    max_length=512
)

print("Fixed training configuration created!")
```

```
Fixed training configuration created!
```

```
from trl import SFTTrainer

trainer = SFTTrainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    processing_class=tokenizer,
    peft_config=lora_config,
)

print("Trainer recreated successfully!")
```

```
/usr/local/lib/python3.13/dist-packages/peft/mapping_func.py:148: UserWarning: You are trying to modify a model with PEFT for the first time. Please make sure you are not using a model that has already been modified by PEFT.
warnings.warn(
/usr/local/lib/python3.13/dist-packages/peft/tuners/tuners_utils.py:331: UserWarning: Already found a `peft_config` attribute in the model. This might lead to unexpected behavior.
warnings.warn(
Trainer recreated successfully!
```

```
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto"
)

model = prepare_model_for_kbit_training(model)

print("Clean Qwen model loaded and prepared!")
```

```
Loading weights: 100% 338/338 [00:09<00:00, 129.37it/s]
Clean Qwen model loaded and prepared!
```

```
trainer = SFTTrainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    processing_class=tokenizer,
    peft_config=lora_config,
)

print("Clean trainer created successfully!")
```

```
Clean trainer created successfully!
```

```
trainer.train()
```

[transformers] The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model [29/250 05:44 < 47:03, 0.08 it/s, Epoch 0.11/1]

Step	Training Loss
10	3.207365
20	3.080478

[250/250 52:30, Epoch 1/1]

Step	Training Loss
10	3.207365
20	3.080478
30	3.002952
40	3.005592
50	2.960402
60	2.938725
70	3.009829
80	2.956402
90	3.016202
100	3.003364
110	2.968521
120	2.970846
130	2.994882
140	2.932876
150	2.978050
160	3.009464
170	2.981423
180	2.989411
190	2.969036
200	2.979305
210	2.942148
220	2.956528
230	2.965348
240	2.982007
250	2.911231

TrainOutput(global_step=250, training_loss=2.988495475769043, metrics={'train_runtime': 3161.9583, 'train_samples_per_second': 0.623, 'train_steps_per_second': 0.623, 'total_flos': 5277617401205760, 'train_loss': 2.988495475769043})

adapter_path = "./qwen_imdb_qlora"

trainer.save_model(adapter_path)

print("QLoRA adapter saved successfully!")

QLoRA adapter saved successfully!

import os

print(os.listdir(adapter_path))

['checkpoint-100', 'chat_template.jinja', 'tokenizer_config.json', 'adapter_model.safetensors', 'checkpoint-250', 'training_log.txt']

```
test_review = """
This movie was absolutely fantastic. The story was engaging,
the actors gave excellent performances, and I really enjoyed
every minute of it.
"""
```

```
prompt = f"""Review:
{test_review}"""
```

```
Sentiment:
"""
```

```
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
```

```

with torch.no_grad():
    outputs = model.generate(
        **inputs,
        max_new_tokens=10
    )

response = tokenizer.decode(outputs[0], skip_special_tokens=True)

print(response)

```

[transformers] `use\_cache=True` is incompatible with gradient checkpointing. Setting `use\_cache=False`.
 [transformers] Caching is incompatible with gradient checkpointing in Qwen2DecoderLayer. Setting `past\_key\_values=None`.
 Review:

This movie was absolutely fantastic. The story was engaging, the actors gave excellent performances, and I really enjoyed every minute of it.

Sentiment:
 Positive.
 :
 :
 :
 :
 :
 :
 :
 :

```

test_review = """
This movie was terrible. The story was boring and predictable,
the acting was poor, and I regretted watching it.
"""

prompt = f"""Review:
{test_review}

Sentiment:
"""

inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

with torch.no_grad():
    outputs = model.generate(
        **inputs,
        max_new_tokens=5
    )

response = tokenizer.decode(outputs[0], skip_special_tokens=True)

print(response)

```

Review:

This movie was terrible. The story was boring and predictable, the acting was poor, and I regretted watching it.

Sentiment:
 Negative av.
 :
 :

```

correct = 0
total = 20

for i in range(total):
    review = test_dataset[i]["text"]

    # Get the original sentiment
    actual = "Positive" if test_dataset[i]["label"] == 1 else "Negative"

    # Extract only the review from our formatted text
    review_text = review.split("Sentiment:")[0].replace("Review:", "").strip()

    prompt = f"""Review:
{review_text}

Sentiment:"""

```

```

inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

with torch.no_grad():
    outputs = model.generate(
        **inputs,
        max_new_tokens=3
    )

generated = tokenizer.decode(
    outputs[0][inputs["input_ids"].shape[1]:],
    skip_special_tokens=True
).strip().lower()

if generated.startswith("positive"):
    predicted = "Positive"
elif generated.startswith("negative"):
    predicted = "Negative"
else:
    predicted = "Unknown"

if predicted == actual:
    correct += 1

print(f"{i+1}. Actual: {actual} | Predicted: {predicted}")

accuracy = correct / total * 100

print("\n" + "=" * 40)
print(f"Correct: {correct}/{total}")
print(f"Accuracy: {accuracy:.2f}%")
print("=" * 40)

```

```

1. Actual: Positive | Predicted: Positive
2. Actual: Positive | Predicted: Positive
3. Actual: Negative | Predicted: Negative
4. Actual: Positive | Predicted: Positive
5. Actual: Negative | Predicted: Negative
6. Actual: Positive | Predicted: Positive
7. Actual: Positive | Predicted: Negative
8. Actual: Negative | Predicted: Negative
9. Actual: Negative | Predicted: Negative
10. Actual: Positive | Predicted: Positive
11. Actual: Positive | Predicted: Positive
12. Actual: Negative | Predicted: Negative
13. Actual: Negative | Predicted: Negative
14. Actual: Negative | Predicted: Negative
15. Actual: Positive | Predicted: Positive
16. Actual: Positive | Predicted: Positive
17. Actual: Negative | Predicted: Negative
18. Actual: Negative | Predicted: Negative
19. Actual: Positive | Predicted: Positive
20. Actual: Positive | Predicted: Positive

```

```

=====
Correct: 19/20
Accuracy: 95.00%
=====

```

```

correct = 0
unknown = 0
predictions = []

model.eval()

for i, example in enumerate(test_dataset):

    # Extract original review
    review_text = example["text"].split("Sentiment:")[0]
    review_text = review_text.replace("Review:", "").strip()

    actual = example["label"]

    prompt = f""Review:
{review_text}

Sentiment:"""

    inputs = tokenizer(
        prompt,
        return_tensors="pt"
    ).to(model.device)

    with torch.no_grad():
        outputs = model.generate(

```

```

        **inputs,
        max_new_tokens=3,
        do_sample=False,
        pad_token_id=tokenizer.pad_token_id
    )

    generated = tokenizer.decode(
        outputs[0][inputs["input_ids"].shape[1]:],
        skip_special_tokens=True
    ).strip().lower()

    if "positive" in generated:
        predicted = 1
    elif "negative" in generated:
        predicted = 0
    else:
        predicted = None
        unknown += 1

    predictions.append(predicted)

    if predicted == actual:
        correct += 1

    if (i + 1) % 50 == 0:
        print(f"Processed: {i + 1}/500")

accuracy = correct / len(test_dataset) * 100

print("\n" + "=" * 50)
print("FINAL EVALUATION")
print("=" * 50)
print(f"Total Reviews : {len(test_dataset)}")
print(f"Correct      : {correct}")
print(f"Unknown       : {unknown}")
print(f"Accuracy      : {accuracy:.2f}%")
print("=" * 50)

```

```

Processed: 50/500
Processed: 100/500
Processed: 150/500
Processed: 200/500
Processed: 250/500
Processed: 300/500
Processed: 350/500
Processed: 400/500
Processed: 450/500
Processed: 500/500

```

```

=====
FINAL EVALUATION
=====
Total Reviews : 500
Correct      : 470
Unknown     : 0
Accuracy     : 94.00%
=====

```

```

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Remove any unknown predictions
valid_indices = [
    i for i, pred in enumerate(predictions)
    if pred is not None
]

actual_labels = [
    test_dataset[i]["label"]
    for i in valid_indices
]

predicted_labels = [
    predictions[i]
    for i in valid_indices
]

# Create confusion matrix
cm = confusion_matrix(
    actual_labels,
    predicted_labels
)

```

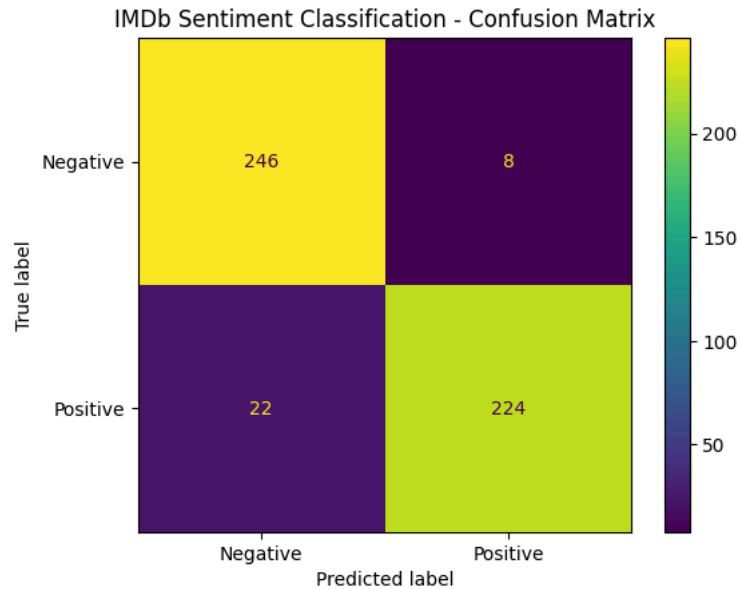
```
print("Confusion Matrix:")
print(cm)

# Display confusion matrix
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Negative", "Positive"]
)

disp.plot()
plt.title("IMDb Sentiment Classification - Confusion Matrix")
```

Confusion Matrix:

```
[[246  8]
 [ 22 224]]
```



```
from sklearn.metrics import classification_report

print(classification_report(
    actual_labels,
    predicted_labels,
    target_names=["Negative", "Positive"]
))
```

	precision	recall	f1-score	support
Negative	0.92	0.97	0.94	254
Positive	0.97	0.91	0.94	246
accuracy			0.94	500
macro avg	0.94	0.94	0.94	500
weighted avg	0.94	0.94	0.94	500

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.